Filosofía Lean - Método Kanban en el contexto del desarrollo de software

La **Filosofía Lean** y el **Método Kanban** forman una combinación clave en la gestión moderna de productos de software. Mientras que Lean proporciona los principios estratégicos y culturales enfocados en maximizar el valor para el cliente, eliminar desperdicios y mejorar continuamente, Kanban es el método que permite materializar y llevar a la práctica esos principios abstractos.

Filosofía Lean en el Desarrollo de Software

Su objetivo principal es maximizar el valor entregado al cliente, eliminar cualquier esfuerzo que no genere valor (considerado desperdicio) y promover la mejora continua.

Los 7 Principios de Lean:

1. **Eliminar desperdicios**: Identificar y erradicar todo aquello que no aporte valor directo al cliente.

2. **Amplificar el aprendizaje**: Promover la comunicación, la experimentación y los ciclos cortos de retroalimentación.

3. **Diferir compromisos / Tomar decisiones tardías**: Postergar decisiones irreversibles hasta el "último momento responsable", cuando se cuente con la máxima información posible.

4. **Entregar lo antes posible**: Reducir los tiempos de entrega, para responder rápidamente a las necesidades cambiantes del cliente y obtener retroalimentación temprana.

5. **Empoderar al equipo**: Brindar autoridad y autonomía a las personas que realizan el trabajo para organizarse y tomar decisiones.

6. **Construir la integridad en el producto**: Asegurar la calidad intrínseca y extrínseca del producto, es decir, incorporar la calidad a lo largo de todo el desarrollo, para prevenir defectos y evitar correcciones costosas.

7. **Ver el todo**: Optimizar el flujo a lo largo de toda la cadena de valor en lugar de suboptimizar etapas o equipos individuales.

---

7 + 1 Desperdicios

1. **Trabajo parcialmente terminado**: Equivale al inventario. Acumula inversión sin generar valor, corre riesgo de obsolescencia y oculta problemas de calidad.

2. **Características extra (*Extra Features*)**: Equivale a la sobreproducción. Desarrollar funcionalidades adicionales que el cliente no necesita o no solicitó. Incrementa innecesariamente la complejidad del código fuente, el costo de mantenimiento y el esfuerzo de prueba.

3. **Proceso extra**: Pasos burocráticos y documentación innecesaria que no aporta valor.

4. **Transferencia de conocimiento**: Equivale al transporte físico. Ocurre cuando el trabajo o la información salta entre personas o equipos. En cada traspaso se pierde contexto, se producen demoras y se degrada el conocimiento.

5. **Cambio de tareas / Multitasking**: Equivale al movimiento en manufactura. Asignar personas a múltiples tareas o proyectos simultáneamente puede ocasionar pérdida de productividad debido a los cambios de contexto.

6. **Esperas / Demoras**: Tiempos muertos producidos al esperar decisiones, aprobaciones,recursos o la disponibilidad de otras personas. Retrasa la llegada de valor al cliente y dificulta la retroalimentación rápida.

7. **Defectos**: Fallas que requieren tiempo y esfuerzo de retrabajo. El costo de un defecto aumenta según el impacto y el tiempo que permanece sin detectarse.

8. **+1. Talento no utilizado**: Consiste en no aprovechar las cualidades del personal (conocimientos, habilidades).

El Método Kanban en el Contexto de Software

Kanban **NO es una metodología** sino un método de gestión del cambio evolutivo y la mejora continua del flujo de trabajo.

Principios Kanban

A. Gestión del Cambio

Reconoce el impacto que provoca el cambio dentro de una organización:

1. **Comenzar con lo que haces ahora**: Implica entender los procesos actuales tal y como se realizan en el presente, respetando roles y responsabilidades existentes.

2. Se busca introducir cambios de forma continua e incremental en lugar de transformaciones drásticas. Esto minimiza la resistencia inicial al cambio.

3. Promover iniciativas de mejora y liderazgo desde cualquier nivel de la organización.

B. Entrega de Servicios

Considera a la organización como una red de servicios enfocada en entregar valor:

1. Enfocarse en cumplir las necesidades y expectativas del cliente.

2. Permitir que las personas se auto-organicen alrededor del flujo de trabajo.

3. Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados.

Las 6 Prácticas Fundamentales de Kanban:

1) Visualizar el flujo de trabajo: Dividir el trabajo en unidades discretas (tarjetas o *kan-bans*) y representar las etapas del proceso en un **tablero Kanban**. Esto hace visible el trabajo y expone dependencias y bloqueos.

2) Limitar el trabajo en progreso (WIP): Limitar la cantidad de tareas permitidas en cada estado del tablero. Esto cambia el paradigma de un sistema que "empuja" tareas (*push system*) a un sistema que las arrastra (*pull system***)**, donde solo se toma nuevo trabajo si queda capacidad disponible.

3) Gestionar el flujo: Medir y optimizar métricas (como *Lead* Time o *Cycle Time)*, identificando y resolviendo cuellos de botella para lograr un flujo suave e ininterrumpido.

4) Hacer las políticas explícitas: Establecer acuerdos claros para todo el equipo sobre los criterios de entrada y salida de cada columna. Las políticas deben acordarse entre todas las partes involucradas y ser expuestas en un lugar destacado.

5) Establecer ciclos de retroalimentación: Implementar reuniones diarias, revisiones de entrega y retrospectivas para evaluar el progreso y la calidad.

6) Mejorar colaborativamente, evolucionar experimentalmente: Implementar cambios incrementales basándose en datos.

Todo esto sostenido sobre los **9 valores de Kanban**: *Transparencia, Equilibrio, Colaboración, Foco en el cliente, Flujo, Liderazgo, Entendimiento, Acuerdo y Respeto*.

¿Cómo aplicar Kanban?

1. **Analizar el proceso actual:** Identificar cómo se trabaja, qué problemas existen y dónde se producen retrasos o cuellos de botella.

2. **Identificar las unidades de trabajo:** Dividir el trabajo en tareas individuales.

3. **Diseñar el tablero Kanban:** Representar visualmente las etapas del proceso mediante columnas.

4. **Definir políticas y prioridades:** Establecer reglas para mover las tareas entre etapas, determinar cuándo una tarea está terminada y clasificar el trabajo según su urgencia.

5. **Establecer límites de WIP:** Limitar la cantidad de tareas que pueden trabajarse simultáneamente en cada etapa para evitar la sobrecarga y favorecer el flujo continuo.

6. **Gestionar y mejorar el flujo:** Supervisar el avance de las tareas, detectar bloqueos y cuellos de botella, y realizar ajustes para mejorar el proceso.

---

3. Sinergia entre Lean y Kanban

Kanban es un método de gestión del trabajo, que proporciona las herramientas operativas para materializar y llevar a la práctica los principios abstractos de Lean.

Eliminar Desperdicios

- **Herramientas y Prácticas Kanban:** Visualizar el flujo de trabajo y Limitar el WIP (*Work In Progress*).
- El tablero Kanban hace visible el trabajo y expone los cuellos de botella, bloqueos y actividades que no agregan valor.

- Limitar el trabajo en curso ataca directamente el desperdicio del "trabajo parcialmente terminado" y reduce la sobrecarga por cambios de contexto (*multitasking*).

Entregar lo Antes Posible

- **Herramientas y Prácticas Kanban:** Sistema de Tracción (*Pull System*) y limitar el WIP.
- El sistema de tracción (*pull system)* permite iniciar nuevas tareas sólo si se cuenta con la capacidad suficiente, evitando demoras ocasionadas por sobrecarga de trabajo.
- Conforme a la Ley de Little, limitar la cantidad de WIP logra una reducción directa en el tiempo de entrega.

Ver el Todo

- **Herramientas y Prácticas Kanban:** Gestión del Flujo.
- En lugar de suboptimizar etapas o evaluar el rendimiento individual de forma aislada, Kanban gestiona el recorrido completo de los ítems de trabajo

Integrar la Calidad

- **Herramientas y Prácticas Kanban:** Hacer Políticas Explícitas.
- Publicar de forma visible las reglas del sistema (como la *Definition of Done*) previene la propagación de defectos.

Amplificar el aprendizaje

- **Herramientas y Prácticas Kanban:** Implementar ciclos cortos de retroalimentación
- Las cadencias (reuniones periódicas de revisión) y el uso de métricas proveen el conocimiento necesario para ajustar el proceso continuamente.

El enfoque de Lean / Kanban: Flujo Continuo y Producto

Lean y Kanban reemplazan la idea de proyecto por el concepto de un flujo continuo de trabajo a lo largo de una cadena de valor:

Flujo continuo, sin inicio ni fin rígidos: En Kanban el trabajo fluye de forma continua sobre un *tablero persistente* que no se limpia ni se reinicia. No existe el concepto de proyecto fijo, sino que los plazos se gestionan por ítem de trabajo según las necesidades.

Gestión de Producto, no de Proyecto: Se gestiona la evolución del producto a lo largo del tiempo. Las solicitudes de los clientes entran al sistema según la demanda (*pull system*) y se procesan *Just in Time*.

Medición del proceso, no del cumplimiento del plan: En lugar de controlar si un "proyecto va a tiempo" según una planificación fija, Lean/Kanban mide la eficiencia y previsibilidad del proceso de entrega (evaluando métricas como *Lead Time* y *Cycle Time*).

Métricas en los 3 enfoques (Tradicional / Lean / Agile)

Las métricas son valores numéricos que representan la presencia o grado de determinados atributos en un proceso, proyecto o producto.

Dominios Generales de las Métricas

Independientemente del enfoque, las métricas se dividen tradicionalmente en tres dominios:

- **Métricas de Proceso**: Evalúan el proceso a nivel organizacional. Son públicas y se utilizan para la mejora continua del proceso a largo plazo.
- **Métricas de Proyecto**: Permiten controlar y gestionar los recursos, el presupuesto, los plazos y el esfuerzo del equipo. Suelen ser privadas del equipo o del proyecto
- **Métricas de Producto**: Enfocadas en las características técnicas de lo que se está construyendo.

Métricas en el Enfoque Tradicional

En el modelo tradicional, se mide de forma exhaustiva abarcando los **tres dominios** (proceso, proyecto y producto) para lograr previsibilidad.

Las métricas básicas son:

- **Esfuerzo**: Horas/hombre invertidas en tareas y el desarrollo.
- **Tiempo / Calendario**: Seguimiento del cronograma y fechas programadas contra fechas reales de entrega.
- **Defectos**: Cantidad de errores detectados por fases, tasa de corrección, densidad y severidad de fallas.
- **Tamaño del Producto**: Mediciones absolutas tradicionalmente como Líneas de Código (LOC), Puntos de Función o Casos de Uso.

Métricas en el Enfoque Ágil

El agilismo percibe la medición como un subproducto del trabajo y no como una actividad aparte que sobrecarga al equipo y establece que el software funcionando es la principal medida de progreso.

Las métricas en el contexto ágil, pertenecen a cada equipo y **no son extrapolables** a otros equipos o proyectos.

Velocidad: Cantidad de *Story Points* completados y aceptados por el *Product Owner* al finalizar un *Sprint*. Mide la estabilidad y el ritmo del equipo, es de uso interno y se utiliza para planificar y predecir, no para comparar equipos o evaluar personas.

Capacidad: Trabajo estimado que el equipo puede asumir en el próximo Sprint. Se estima en base a la disponibilidad, en horas-hombre o puntos de historia.

Running Tested Features (RTF): Cantidad de funcionalidades probadas, funcionando e integradas en el sistema. Refleja el software funcionando.

Métricas en el Enfoque Lean / Kanban

En Lean y Kanban, el foco principal de medición es el **Proceso y el Flujo de Trabajo**, analizando el tiempo que demoran los ítems de trabajo en atravesar el sistema continuo.

Lead Time: Perspectiva del cliente. Mide el tiempo que transcurre desde que el cliente solicita un ítem de trabajo hasta que este se entrega. Se suele medir en días de trabajo.

Cycle Time: Perspectiva interna del equipo, donde se registra el tiempo que transcurre desde que el equipo comienza a trabajar en un ítem de trabajo hasta que se entrega. Es una medición mecánica de la capacidad del proceso en donde se utilizan días de trabajo o esfuerzo.

Touch Time: Tiempo neto real de trabajo sobre un ítem o tarea, excluyendo tiempos de espera y bloqueos.

Eficiencia del Proceso: Proporción de tiempo que agrega valor. Se calcula como (*Touch Time / Elapsed Time) * 100*. El tiempo transcurrido suele ser el *Cycle Time*.

Cuadro Resumen Comparativo

Enfoque	Foco de Medición Principal	Métricas Clave

Tradicional	Proyecto, Proceso y Producto (Previsibilidad)	Esfuerzo, Tiempo/Calendario, Costos, Defectos, Tamaño (LOC/Casos de Uso).
Ágil (Scrum)	Producto / Software Funcionando	Velocidad, Capacidad, Running Tested Features (RTF).
Lean (Kanban)	Flujo de Trabajo / Proceso Continuo	Lead Time, Cycle Time, Touch Time, Eficiencia de Ciclo.

Gestión de software como producto - Prácticas Continuas.

En la gestión moderna de desarrollo, la **gestión de software como producto** plantea un cambio de paradigma respecto a la gestión tradicional de **proyectos**:

Enfoque en Proyecto vs. Enfoque en Producto: Mientras que los marcos a nivel de proyecto (como Scrum) gestionan iteraciones, calendarios y dinámicas temporales de equipos, la gestión del software como **producto** trasciende la duración de un proyecto individual y se enfoca en la evolución, integridad, mantenibilidad y calidad intrínseca del producto a lo largo de todo su ciclo de vida.

Prácticas Continuas

Integración Continua (*Continuous Integration - CI*)

- Los desarrolladores integran sus cambios de código en un repositorio compartido de forma frecuente.
- Cada commit o integración activa automáticamente un proceso de *build* y la ejecución de una suite de pruebas (unitarias y de integración).
- Su propósito es detectar errores de integración de manera temprana, evitando inconsistencias al final del ciclo y garantizando que el sistema compile con calidad.

Entrega Continua (*Continuous Delivery - CD*)

- Disciplina que asegura que el producto de software se construya de forma tal que siempre esté en condiciones de ser liberado a producción.
- Automatiza no solo las pruebas unitarias sino también las pruebas de aceptación y los procesos de empaquetado y despliegue.
- Aunque el artefacto esté listo para desplegarse, la decisión final sigue siendo responsabilidad del *Product Owner* o el área correspondiente.
- Requiere Integración Continua previa.

Despliegue Continuo (*Continuous Deployment*)

- Extensión de la Entrega Continua donde se elimina totalmente la intervención humana para el pase a producción.
- Todo cambio en el código que supere con éxito los *pipelines* automatizados de integración y prueba, se despliegan automáticamente en el ambiente de producción.
- Para mitigar el riesgo de fallas se utilizan técnicas automatizadas como Canary Deployments (despliegue gradual a un pequeño grupo de usuarios) y Blue/Green Deployments (alternancia entre dos entornos idénticos o equivalentes).

Testing Continuo y Prácticas Asociadas: Para garantizar que la velocidad de entrega continua no degrade la calidad del producto, se integran prácticas continuas de prueba y refactorización:

- **Test-Driven Development (TDD)**: Técnica de desarrollo donde se escribe primero la prueba unitaria (*Test-First*), se codifica la solución mínima para que pase dicha prueba y luego se realiza *Refactoring* para mantener el código limpio sin alterar su comportamiento.
- **Acceptance Test-Driven Development (ATDD)**: Desarrollo guiado por pruebas de aceptación para validar los criterios del cliente antes de implementar la funcionalidad.
- **Pruebas de Regresión Automatizadas**: Ejecución continua de suites de prueba a nivel de sistema para confirmar que nuevos cambios no rompan funcionalidades previas.
- **Versionado Integrado**: El código fuente, los casos de prueba automatizados y los scripts de configuración se versionan conjuntamente en el mismo repositorio.

Práctica	Qué es	Clave
CI	Integrar código varias veces al día	Cada commit dispara build + pruebas automáticas; detecta errores temprano

Entrega continua	El software siempre está listo para liberarse	Requiere CI; automatiza pruebas de aceptación y despliegue; **la liberación es manual** (decisión del negocio)
Despliegue continuo	Todo cambio que pasa el pipeline va a producción	**Sin intervención humana**; se mitiga con Canary y Blue/Green
Testing continuo	Mantener la calidad con la velocidad	TDD, ATDD, regresión automatizada, versionado integrado

Conceptos generales sobre calidad.

La **calidad en el software** se define como el cumplimiento de los requerimientos funcionales y de desempeño explícitamente definidos, los estándares documentados y las características implícitas esperadas.

Principios de la Calidad

- **La calidad no se "inyecta" al final:** No se puede agregar calidad al producto en etapas finales de manera artificial sino que debe estar embebida desde el inicio
- **Es responsabilidad de todos:** No es una tarea exclusiva, requiere la colaboración de todo el equipo y el patrocinio de la gerencia
- **Enfoque preventivo en lugar de correctivo**: Prevenir defectos en etapas tempranas es menos costoso que hacerlo en etapas avanzadas o finales
- **Lo que no se mide, no se controla:** Medir la calidad requiere del uso de indicadores y métricas
- **Más pruebas no implica necesariamente mayor calidad:** Sobre todo si los procesos previos fueron deficientes

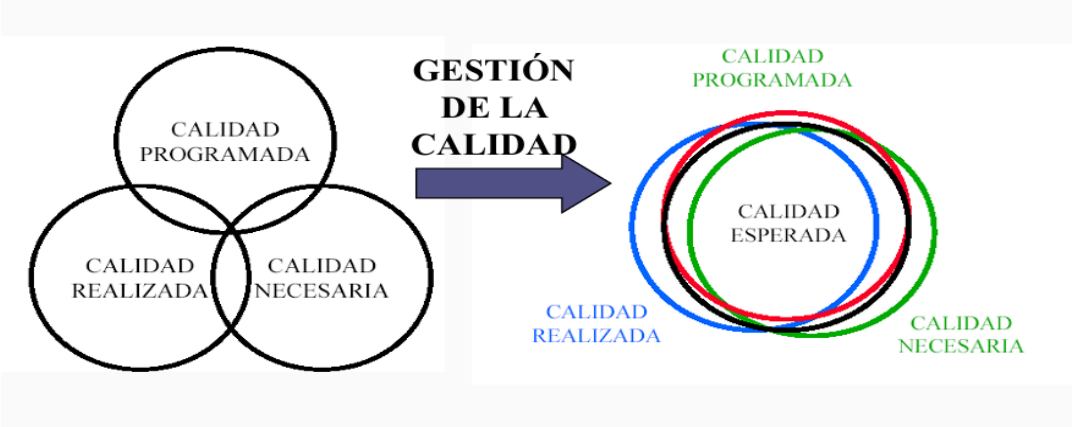
Visiones y Perspectivas de la Calidad

- **Visión del Usuario:** Centrada en la usabilidad y el cumplimiento de sus necesidades
- **Visión del Producto:** Valuada mediante el cumplimiento de estándares y especificaciones técnicas
- **Visión del Proceso (Manufactura):** Enfocado en que el proceso de desarrollo sea el adecuado y genere valor real, eliminando actividades triviales
- **Visión del Valor:** Busca el equilibrio óptimo entre la relación costo-beneficio, tanto para la organización como para el cliente

El Trinomio de la Gestión de Calidad

Para lograr la **Calidad Esperada**, la gestión de calidad busca alinear tres dimensiones:

- **Calidad Programada**: Los alcances y características planificadas en el proyecto.
- **Calidad Realizada**: Lo que efectivamente se construyó e implementó.
- **Calidad Necesaria**: Las características mínimas reales que el cliente y el negocio requieren.



Toda desviación fuera de la coincidencia de estas tres esferas se convierte en desperdicio o insatisfacción.

Calidad de Proceso vs. Calidad de Producto

- **Calidad de Proceso**: Las características del cliente, el negocio y la tecnología son variables externas difíciles de controlar. Así, el proceso de desarrollo es el único factor verdaderamente controlable que permite mejorar de forma predecible la calidad de los entregables.
- **Calidad de Producto**: Evalúa el artefacto resultante. Como los requerimientos funcionales varían entre proyectos, la calidad del producto se mide y se compara a través de sus atributos de calidad, en su mayoría requerimientos no funcionales, mediante modelos estandarizados.

Importancia de trabajar para y con Calidad. Ventajas y Desventajas.

Trabajar **para y con calidad** implica asumir un enfoque preventivo a lo largo de todo el proceso de desarrollo, cumpliendo con los requerimientos funcionales y de rendimiento explícitamente definidos, los estándares documentados y las características implícitas esperadas.

Importancia de trabajar con y para Calidad

- **Previene el fracaso habitual de los proyectos**: Sin gestión de calidad, los proyectos frecuentemente sufren retrasos, costos excedidos, no alcanzan sus objetivos o son cancelados.
- **Es más barato trabajar con calidad que sin ella**: Integrar la calidad en el desarrollo ayuda a prevenir y detectar errores tempranamente, lo que reduce drásticamente costos de corrección y retrabajo
- **Ventaja competitiva**: La calidad actúa como elemento diferenciador que permite destacar de la competencia, retener clientes y mantener la competitividad en el mercado.
- **Satisfacción integral de las partes interesadas**: La calidad no solo satisface las necesidades y expectativas del cliente, sino también la del usuario final, la gerencia y del equipo de desarrollo.

Ventajas de trabajar con Calidad

- **Ahorro de costos**: Prevenir termina siendo menos costoso que corregir, ya que se evita incurrir en costos provocados por fallas y reduce el retrabajo.
- **Mayor confianza y retención de clientes**: La entrega frecuente de software confiable y de calidad, mejora la reputación y fideliza a los clientes.
- **Previsibilidad y reducción de riesgos**: Facilita controlar plazos y costos, reduciendo retrasos, sobrecostos y la cancelación del proyecto.
- **Ventaja competitiva**: La calidad es un elemento diferenciador que permite destacar de la competencia.
- **Mejor clima laboral**: Facilita la reutilización y el mantenimiento, reduce la deuda técnica y las urgencias por fallos en producción, y mejora el clima del equipo.

Desventajas, Desafíos y Costos Asociados

Trabajar con calidad no tiene como desventaja principal la calidad en sí, sino que se asocia más a los esfuerzos y costos necesarios para implementarla.

- **Esfuerzo e inversión inicial**: Adoptar disciplinas de calidad exige tiempo, inversión y capacitación inicial.

- **Sobrecarga de costos en etapas tempranas**: Las inspecciones y revisiones pueden ralentizar el inicio del proyecto y generar la idea de un costo adicional, aunque se compensa ampliamente en el futuro.
- **Resistencia al cambio**: Exige compromiso y un cambio cultural profundo, lo cual requiere esfuerzo y gestión del cambio.
- **Riesgo de burocracia o sobre-ingeniería**: Si los estándares y procesos de calidad no no se adaptan correctamente al negocio, pueden demorar y volver rígido el desarrollo.
- **No garantiza la ausencia de errores**: Las pruebas reducen el riesgo, pero no aseguran la ausencia total de defectos, en especial si las etapas anteriores fueron deficientes.

«¿Por qué es importante trabajar para y con calidad? ¿Cuáles son sus ventajas y desventajas?»

Trabajar para y con calidad es importante porque permite prevenir defectos durante todo el ciclo de desarrollo, en lugar de corregirlos cuando ya son costosos. Esto reduce el retrabajo, los costos y los riesgos, y permite obtener un software que satisfaga las necesidades de los clientes, los usuarios y el equipo.

Sus principales ventajas son el ahorro económico a largo plazo, la satisfacción del cliente, la previsibilidad de los proyectos, la ventaja competitiva y una mayor facilidad de mantenimiento.

Sus desafíos son la inversión inicial, el tiempo dedicado a las revisiones y pruebas, la resistencia al cambio y el riesgo de generar burocracia. Además, trabajar con calidad no garantiza eliminar todos los errores. Sin embargo, los beneficios suelen compensar el esfuerzo inicial cuando las prácticas de calidad se aplican de manera adecuada.

Actividades relacionadas con el Aseguramiento de Calidad del Software (SQA).

Administración de la calidad del software

Conjunto de actividades que se encargan de definir, planificar, evaluar y controlar la calidad del proceso de desarrollo y del producto de software resultante a lo largo de todo el ciclo de vida

- Asegurar el cumplimiento de los niveles de calidad requeridos.

- Establecer estándares y procesos de calidad apropiados y velar por que se respeten en la práctica.
- Promover una cultura de calidad preventiva, donde la calidad sea responsabilidad de todos y cada uno de los miembros del equipo.

Estas actividades se estructuran en tres grandes componentes: **Aseguramiento de Calidad**, **Planificación de Calidad** y **Control de Calidad**:

Aseguramiento de Calidad (SQA)

Conjunto de actividades planificadas y sistemáticas que se aplican a lo largo de todo el ciclo de vida con el fin de garantizar que los procesos y los productos cumplen con los estándares y requisitos definidos. Posee un enfoque preventivo y orientado al proceso y suele realizarse con independencia del equipo de desarrollo.

Sus actividades principales son:

- Desarrollar los métodos, herramientas y modelos de calidad que se utilizarán como referencia para comparar y evaluar
- Verificar el cumplimiento de los estándares y procesos definidos a través de auditorías, revisiones e inspecciones.
- Reportar y dar seguimiento a las no conformidades hasta su corrección.

Planificación de la Calidad

Actividad realizada al inicio del proyecto que define cómo se alcanzará y verificará la calidad requerida, adaptando la estrategia de calidad al contexto específico del proyecto.

- **Elaborar el Plan de SQA**: Se seleccionan y se adaptan los estándares y procedimientos según las particularidades del proyecto. El plan también define roles, responsabilidades, actividades de calidad y el tratamiento de las no conformidades.
- **Definir metas de calidad**: Se identifica qué productos deben evaluarse, cuáles son sus atributos de calidad más significativos y con qué mecanismos serán valorados.

Control de Calidad (QC)

Conjunto de actividades orientadas a evaluar el producto (y su proceso de construcción) para detectar defectos y verificar que se cumplen los requisitos y estándares definidos.

Comprende dos enfoques principales:

- **Revisiones**: Un grupo examina el artefacto o proceso junto con su documentación para identificar potenciales problemas. Se divide en:
    - Inspecciones, para remoción de defectos
    - Revisiones de progreso, para evaluar el avance respecto al plan
    - Revisiones de calidad, para verificar el cumplimiento de estándares
- **Evaluaciones Automáticas y Mediciones**: Pruebas de software (*testing*) y análisis cuantitativos sobre el producto construido.
    - Testing de Software: Proceso con mentalidad destructiva orientado a encontrar defectos cuya presencia se asume de antemano. Consiste en someter al código a condiciones específicas para demostrar si cumple con los requerimientos. Puede demostrar la presencia de defectos pero no su ausencia.

Principales Modelos de Calidad existentes (CMMI – SPICE – ISO).

Los modelos de calidad son marcos de referencia que establecen criterios, características y objetivos de calidad para evaluar procesos y productos de software, permitiendo que la organización determine cómo implementar las prácticas necesarias para cumplirlos.

Propósito: evaluar y mejorar la calidad de los productos, así como la capacidad y madurez de los procesos de desarrollo.

¿Qué es CMMI?

CMMI es un modelo de referencia empírico y descriptivo para la mejora y evaluación de procesos en el desarrollo, mantenimiento y operación de sistemas de software. No es un estándar ni una norma rígida, tampoco se certifica, sino que las organizaciones acreditan su nivel de madurez mediante una evaluación objetiva, llevada a cabo por un *Lead Appraiser*

No prescribe una única forma de trabajar: establece buenas prácticas y permite evaluar qué tan bien se implementan los procesos.

¿Para qué sirve?

- Evaluar la madurez y capacidad de los procesos.
- Identificar fortalezas y debilidades.
- Guiar la mejora continua.
- Reducir riesgos y mejorar la previsibilidad de los proyectos.

Las 3 Constelaciones de CMMI

- **CMMI-DEV (Desarrollo)**: Ofrece guías para medir, monitorear y administrar los procesos de desarrollo de software.
- **CMMI-ACQ (Adquisición)**: Provee lineamientos para seleccionar, administrar y adquirir productos y servicios de terceros.
- **CMMI-SVC (Servicios)**: Provee guías para la entrega de servicios internos o externos.

Representaciones de CMMI-DEV:

Por Etapas: Evalúa la **madurez organizacional** mediante cinco niveles progresivos.

1. **Inicial:** Procesos impredecibles, reactivos y poco controlados
2. **Administrado:** Los proyectos planifican, ejecutan y controlan procesos
3. **Definido:** Los procesos están estandarizados y caracterizados a nivel organizacional
4. **Cuantitativamente Administrado:** Los procesos se miden y controlan cuantitativamente
5. **Optimizado:** Foco en la mejora continua e innovación tecnológica

Continua: Evalúa la **capacidad** individual de cada Área de Proceso (AP) en lugar de medir a la organización en su conjunto. Utiliza una escala de 4 niveles (CMMI v1.3):

- **Incompleto**
- **Realizado**
- **Administrado**
- **Definido**

Cuadro comparativo

Aspecto	Por etapas	Continua
Foco	Madurez organizacional	Capacidad de cada Área de Proceso
Niveles	5	4
Estructura	Conjunto predefinido de AP por nivel	Cada AP se evalúa por separado
Progresión	Acumulativa y jerárquica	Libre, según el interés del negocio
Beneficio clave	Comparación entre organizaciones	Optimizar áreas críticas con menor esfuerzo inicial
Mejora	Provee una secuencia probada de mejoras	Permite elegir el orden de las mejores según las necesidades

CMMI en ágil

¿Pueden coexistir? Sí. CMMI es un modelo **descriptivo** que no impone prácticas específicas, así que se puede cumplir con prácticas ágiles, aunque ambos lados deben ceder:

Cede la agilidad	Cede CMMI
Trazabilidad de Requerimientos: CMMI exige mantener un control y trazabilidad de los requerimientos en todo momento. Esta trazabilidad puede gestionarse mediante el Backlog y las US	Acepta que los procesos se definan de manera liviana
Auditorías de Calidad: los equipos ágiles deben aceptar las auditorías objetivas e independientes realizadas por alguien externo al equipo	No exige documentación ni registros burocráticos sobre las reuniones diarias

Por niveles de madurez

- **Nivel 1:** alineación básica (se identifica el alcance y se ejecuta).
- **Nivel 2: alta compatibilidad** (planificación, requerimientos, monitoreo). **Fricción:** PPQA, porque pide verificación independiente y la retrospectiva es una revisión interna.
- **Niveles 3 a 5: conflicto creciente.** CMMI pide procesos estándar organizacionales y control cuantitativo y estadístico. Ágil sostiene que la experiencia de un proyecto no se extrapola a otros contextos cambiantes.

CMMI vs. ágil

Criterio	CMMI	Ágil
Mejora	Organizacional	Por proyecto
Naturaleza	Descriptivo y cuantitativo	Prescriptivo y cualitativo (conocimiento tácito)
Éxito	Predictibilidad y estabilidad	Rendimiento y velocidad
Conocimiento	Activos de proceso estandarizados	Centrado en las personas
Comunicación	Estructurada, organizacional	Directa, persona a persona
Atajos	Desalentados	Alentados (sacar burocracia)
Riesgos	Proactiva (planificación formal)	Adaptativa (inspección y adaptación)

Similitudes

- Misma meta: buscan crear organizaciones de **alto desempeño**.
- Ambos **planifican** el trabajo
- Tienen **reglas** cuyo incumplimiento tiene consecuencias.
- Ambos se basan en la **experiencia**.
- **Contextuales,** no pueden aplicarse ciegamente a cualquier proyecto.

SPICE (ISO 15504)

SPICE es un modelo normativo/teórico creado para la evaluación y mejora de la capacidad de los procesos de software.

- **Niveles de Madurez/Capacidad**: Clasifica a las organizaciones o procesos en **6 niveles** (desde *Nivel 0 - Organización inmadura* hasta *Nivel 5 - Organización en optimización*).
- **Naturaleza Dual y Modelo IDEAL**: Es un modelo dual que combina un modelo de calidad con un modelo de ciclo de mejora como *IDEAL* para guiar la ejecución de proyectos de mejora en las organizaciones.

¿Qué es IDEAL?

IDEAL es un modelo que guía la mejora de procesos organizacionales. Su nombre representa cinco etapas:

- **I** — Initiating: iniciar la mejora.
- **D** — Diagnosing: diagnosticar la situación actual.
- **E** — Establishing: establecer objetivos y un plan de mejora.
- **A** — Acting: ejecutar las acciones planificadas.
- **L** — Learning: aprender de los resultados y realizar ajustes.

La diferencia es que SPICE permite evaluar la capacidad de los procesos, mientras que IDEAL organiza las actividades para mejorarlos.

Normas y Estándares ISO Relacionados

Dentro del ámbito de la calidad se distinguen tanto normas de proceso como de producto:

- **ISO 9001**: Norma internacional enfocada en los sistemas de gestión de calidad de la organización.
- **ISO/IEC 15504 (SPICE)**: Estándar para la evaluación de la capacidad de los procesos de software.
- **ISO/IEC 25010 (SQuaRE / ISO 25000)**: Enfoque orientado a la **calidad de producto**, que evalúa los requerimientos no funcionales a través de atributos como desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad.

Norma	¿En qué se enfoca?
ISO 9001	Sistema de gestión de calidad de la organización.
ISO/IEC 15504 (SPICE)	Evaluación de la capacidad de los procesos.
ISO/IEC 25010 (SQuaRE)	Modelo para especificar, medir y evaluar la calidad del producto de software.

Auditorías al Software

Una **Auditoría de Software** es una **evaluación objetiva e independiente** que se realiza sobre los procesos o productos de software para verificar el cumplimiento de estándares, procedimientos, especificaciones y lineamientos documentados. Es llevada a cabo por un auditor o grupo evaluador externo al equipo de desarrollo para garantizar su imparcialidad.

Beneficios: Detectar desviaciones, mejorar la calidad y brindar información objetiva a la gerencia.

---

1. Auditorías de Proyecto

Verifican que el proyecto se esté ejecutando de acuerdo con el proceso de desarrollo definido y se realiza conforme a lo establecido en el plan de SQA.

- **Enfoque según la metodología**:
    - **Metodologías tradicionales**: la auditoría la realiza una figura objetiva e independiente del equipo del proyecto.
    - **Marcos ágiles**: en general no priorizan las auditorías formales externas, por considerarlas burocráticas; el equipo analiza y ajusta su proceso de forma interna, principalmente durante la *Sprint Retrospective*.

---

2. Auditorías al Grupo de Calidad (GAC)

Se centran en verificar la efectividad, desempeño y objetividad del propio Grupo de Aseguramiento de Calidad (GAC).

- Para que las evaluaciones de calidad no se vean desvirtuadas, la estructura organizacional debe procurar independencia:
    - La administración de calidad debe estar **separada de la administración de proyectos**.
    - El Grupo de Calidad **no debe reportar al Gerente del Proyecto**.
    - Debe depender directamente de la **Gerencia** o la alta dirección para garantizar imparcialidad.
- **Objeto de la Auditoría al Grupo de Calidad**:
    - Evaluar si el GAC cumple adecuadamente sus funciones.
    - Medir la eficiencia de las auditorías mediante **métricas** como el esfuerzo por auditoría, la duración, etc.

Auditorías de Configuración

1. **Física**: Compara el código construido con su documentación técnica de soporte. Asegura que la documentación entregable describa correctamente al código desarrollado y que los ítems de configuración respeten el esquema de nombrado y ubicación.
2. **Funcional**: Compara el software ejecutable y su documentación contra los requerimientos especificados en la ERS. Su objetivo es asegurar que el código implemente **única y totalmente** las capacidades funcionales descritas en los requerimientos

---

1. Roles y Responsabilidades Principales

- **Auditado**: parte auditada (proyecto, área o equipo) representada generalmente por el Líder de Proyecto u otro integrante designado. Solicita la auditoría, acuerda el cronograma, suministra documentación y evidencias, responde consultas, analiza las desviaciones, propone el plan de acción correctivo y es el responsable de implementarlo.
- **Auditor**: Persona o grupo evaluador independiente (externo al proyecto o al área auditada). Planifica y prepara la auditoría, acuerda el alcance, analiza la evidencia, realiza entrevistas, completa las listas de chequeo, redacta el informe, comunica hallazgos y hace seguimiento del plan de acción.
- **Gerente de SQA**: Planifica la estrategia global de auditorías, calcula costos, asigna auditores, resuelve desacuerdos o no conformidades entre auditor y auditado y recibe los informes para reportar a la gerencia.

---

2. Las 4 Etapas del Proceso de Auditoría

Etapa 1: Preparación y Planificación

1. El auditado solicita la auditoría y, junto con el auditor, acuerdan los detalles, como la fecha, el lugar, el alcance, los objetivos y los participantes.
2. El auditor solicita y examina documentación básica antes de la reunión formal.
3. Se establecen las métricas de la auditoría.

Etapa 2: Ejecución

1. El auditor presenta al equipo evaluador, repasa el cronograma, explica el proceso, solicita colaboración y aclara dudas con los auditados.
2. El auditor realiza preguntas y solicita evidencias objetivas para contrastar lo que el equipo dice que hace frente a lo formalmente definido.

Etapa 3: Análisis y Reporte de Resultados

1. El auditor revisa las notas, los *checklists* y la documentación recopilada para realizar una verificación cruzada de las desviaciones frente al proceso.
2. Se presenta un informe preliminar al auditado, se discuten los hallazgos y se acuerda la fecha de entrega del reporte definitivo.
3. El auditor actualiza el documento preliminar con las observaciones de la reunión de cierre, guarda copia y envía el reporte final al auditado.

Etapa 4: Seguimiento

1. El auditado analiza las desviaciones reportadas, redacta un Plan de Acción con fechas de compromiso y lo envía al auditor para su revisión y aprobación.
2. El auditado ejecuta las correcciones e informa su cumplimiento; el auditor verifica objetivamente que las soluciones hayan sido efectivas, da por cerradas las desviaciones e informa la finalización del proceso.

---

3. Técnicas, Herramientas y Hallazgos

Herramientas de Evaluación:

- **Checklists**: Cuestionarios estandarizados de mínima que aseguran consistencia en la evaluación.
- **Muestreo**: Selección representativa de componentes o registros a inspeccionar.
- **Revisión de Registros y Herramientas Automatizadas**: Examen documental y soporte automatizado de recolección de evidencia.

Tipos de Hallazgos en el Reporte:

- **Buenas Prácticas**: Procedimientos y operaciones ejecutadas de forma efectiva o superadora a lo esperado.

- **Observaciones**: Comentarios o sugerencias de mejora no críticas. No requieren un plan de acción
- **Desviaciones / No Conformidades**: Incumplimientos explícitos a estándares, procedimientos o requerimientos. Requieren un plan de acción por parte del auditado

Métricas Típicas de Auditoría:

- Esfuerzo por auditoría
- Duración de auditoría
- Cantidad de desviaciones encontradas y su clasificación según áreas de proceso (ej. CMMI).

Calidad de Producto.

1. Planificación de Pruebas para el Software (*Test Planning*)

El testing es un **proceso destructivo** donde **se trata de encontrar defectos**, cuya presencia se asume de antemano. Su propósito principal es mejorar la calidad mediante la detección temprana de defectos, lo cual ahorra esfuerzo, tiempo y recursos. Sin embargo, **el testing por sí solo NO puede asegurar la calidad al final si esta no fue construida a lo largo del proceso**.

SQA vs QC

La principal diferencia entre ambos radica en su enfoque y objetivo. Por una lado el SQA está orientado al proceso y es de carácter preventivo mientras que el QC está enfocado en el producto y es de carácter detectivo.

Criterio	QA / SQA (Aseguramiento de la Calidad)	QC / SQC (Control de la Calidad)

Enfoque	Preventivo: evita defectos.	Detectivo: identifica defectos.
Objeto	Procesos de desarrollo.	Productos y artefactos.
Objetivo	Mejorar los procesos y prevenir errores.	Detectar errores y verificar requisitos.
Momento	Durante todo el ciclo de vida.	Durante y después de la construcción.
Actividades	Definir procesos, estándares y modelos de calidad.	Revisiones técnicas, auditorías y testing.

Fases del Proceso de Pruebas:

1. **Planificación y Control**: Se define el **Plan de Pruebas** (*Test Plan*), especificando los objetivos,estrategia de pruebas, recursos, criterios de aceptación y los entornos a emular.
2. **Análisis y Diseño**: Se evalúa si los requerimientos son testeables, se identifican los datos y se diseñan los **Casos de Prueba (CP)**.
3. **Ejecución**: Se ejecutan los casos de prueba en los ambientes correspondientes, comparando los resultados obtenidos con los esperados y registrando los incidentes encontrados.
4. **Análisis de Fallas, Evaluación y Reportes**: Se elabora el reporte para los desarrolladores y el **informe final de pruebas** con métricas y estadísticas.

---

2. Niveles de Pruebas de Software

Los niveles de prueba determinan el **foco y la granularidad** del testing, probando desde los componentes más pequeños de forma independiente hasta la integración total del sistema.

Los niveles responden a dónde y cuándo se prueba mientras que los tipos responden a qué se evalúa.

Pruebas Unitarias

- La realizan los desarrolladores sobre un componente, clase o función de forma aislada.
- Requieren acceso al código fuente y se apoyan en *frameworks* de pruebas o depuradores.
- Los errores suelen corregirse en el momento, sin registro formal.

Pruebas de Integración

- Verifican el funcionamiento y comunicación entre **dos o más componentes** que interactúan mediante sus interfaces (*boundaries*).
- Se comprueba que aquellos módulos o componentes que funcionan bien de forma aislada también lo hagan en conjunto.
- Suelen ejecutarse de forma incremental (enfoques *Top-Down* o *Bottom-Up, aunque lo ideal es utilizar ambos*).
- Puede utilizarse una estrategia conocida como *Big Bang,* la cual consiste en integrar todo junto, pero al no ser incremental dificulta ubicar la falla

Pruebas de Sistema

- Evalúan la versión completa e integrada del producto o incremento en un ambiente de testing que emula la operación real.
- Por lo general es ejecutada por un equipo de testing independiente y cubre aspectos funcionales y no funcionales

Pruebas de Aceptación

- Validan que el software satisfaga las necesidades y los requerimientos reales del negocio y del usuario.

- Se ejecutan habitualmente en entornos de **preproducción** antes de pasar a producción.

---

3. Tipos de Pruebas de Software

Se clasifican según el **aspecto u objetivo** que se desea evaluar. Un mismo tipo puede aplicarse en distintos niveles.

Testing Funcional

Evalúa **"qué"** hace el sistema en base a las especificaciones funcionales y las reglas de negocio. Incluye pruebas basadas en **Requerimientos** (criterios de aceptación de historias de usuario) y en **Procesos de Negocio**.

Testing No Funcional

Evalúa **"cómo"** opera el sistema. Incluye pruebas de atributos clave de calidad como:

- *Desempeño(performance):*
    - Carga: comportamiento de la demanda esperada
    - Estrés: más allá de los límites
    - Volumen: grandes cantidades de datos
- Seguridad, usabilidad, fiabilidad, compatibilidad y mantenibilidad.

Smoke Test (Prueba de Humo)

- Ejecución rápida sobre las funcionalidades básicas para comprobar que no existan fallas bloqueantes antes de iniciar un ciclo formal de testing.
- Suele utilizarse como criterio de entrada
- El *Sanity Test* es similar, pero acotado a verificar un cambio o corrección puntual

Pruebas relacionadas con cambios

- **Pruebas de confirmación**: verifican que un defecto ya corregido efectivamente no se repite.

- **Pruebas de regresión**: verifican, frecuentemente de forma automatizada, que las modificaciones o correcciones no hayan introducido errores en funcionalidades previamente validadas.

---

4. Técnicas y Herramientas para Probar Software

Dado que el tiempo y el presupuesto son limitados, las estrategias buscan cubrir la **mayor cantidad de funcionalidades con la menor cantidad de casos de prueba**. El mejor enfoque consiste en combinar múltiples técnicas:

A. Estrategias y Técnicas de Diseño de Casos de Prueba

1. **Técnicas de Caja Negra (Basadas en Especificaciones)**:
    - Evalúan el software como una "caja cerrada", analizando solamente las entradas y salidas, sin mirar el código interno.
    - **Partición de Equivalencias**: Divide los valores de entrada y salida en clases que el sistema trata como equivalentes, probando un valor representativo de cada clase.
    - **Análisis de Valores Límites**: se enfoca en los límites de cada clase, donde los errores son más frecuentes (mínimo, máximo y valores inmediatamente fuera de ellos).
    - **Tabla de decisión**: combina condiciones y acciones para reglas de negocio con varias condiciones.
    - **Basadas en la Experiencia**: *Adivinanza de Defectos* (intuición basada en errores comunes pasados) y *Testing Exploratorio* (diseño y ejecución simultánea guiada por la creatividad y el aprendizaje en vivo).
2. **Técnicas de Caja Blanca (Transparentes)**:
    - Se basan en el análisis directo de la estructura y lógica interna del código fuente.
    - Miden el grado de **cobertura (*coverage*)**, evaluando cobertura de sentencias/líneas, cobertura de decisiones/ramas (ramas *True* y *False*) y cobertura de condiciones.
3. **Caja Gris**:

    - Enfoque intermedio donde se evalúan entradas y salidas pero aprovechando cierto conocimiento interno de la arquitectura o base de datos.

B. Prácticas y Herramientas Tecnológicas

- **Desarrollo Guiado por Pruebas (TDD)**: Técnica orientada a escribir primero el test unitario (*Test-First*), implementar el código mínimo para que la prueba pase y luego refactorizar.
- **Automatización de Pruebas**: Herramientas que ejecutan scripts de prueba para reducir costos y aumentar la frecuencia de ejecución en pruebas de regresión.
- **Herramientas de Combinatoria y Soporte**: Frameworks de pruebas unitarias (como JUnit o NUnit), herramientas de combinación por pares (*Pairwise* / CTweb), entornos de integración continua (CI) y depuradores.

Verificación y Validación (V&V)

Disciplina para evaluar y garantizar la calidad del software durante **todo el ciclo de vida**.

	Verificación	**Validación**
Pregunta	¿Construimos el producto **correctamente**?	¿Construimos el producto **correcto**?
Objetivo	Que los artefactos cumplan las especificaciones y estándares (líneas base).	Que el producto satisfaga las necesidades reales del usuario.
Enfoque	Correspondencia técnica con los productos predecesores.	Utilidad real y propósito del usuario.

Principios

1. **Abarca todo el ciclo de vida:** desde los requerimientos hasta las pruebas de aceptación.
2. **Debe planificarse:** al ser un proceso exigente en términos de tiempo y presupuesto
3. **Involucramiento temprano y economía del defecto:** detectar y corregir fallas en etapas avanzadas es más costoso que hacer en etapas iniciales

Dos aproximaciones complementarias

	Revisiones técnicas	**Testing**
Tipo de análisis	**Estático** (no requiere ejecutar código)	**Dinámico** (exige ejecutar el sistema)
Sobre qué	Requerimientos, diseño, código fuente, planes de prueba	Ejecutables, módulos, servicios, interfaces
Técnicas	Inspecciones, recorridas, listas de chequeo	Pruebas unitarias, de integración, de sistema y de aceptación

Clasificación de hallazgos

- **Error:** fallo o equivocación en el producto de trabajo que se está inspeccionando.
- **Defecto:** falla heredada de un producto de trabajo predecesor no detectada a tiempo.
- **Falla:** comportamiento erróneo durante la ejecución del sistema.

Severidad	**Descripción**
Mayor	Causa una falla operacional o impide cumplir un requerimiento clave.

Menor	Viola estándares sin causar fallas, pero afecta la mantenibilidad.
Cosmética	Errores de tipeo, gramática o plantilla.

Técnicas y Herramientas para la realización de revisiones técnicas del software.

Las **revisiones técnicas** constituyen un proceso de **Verificación y Validación (V&V) estático** llevado a cabo entre pares o colegas, que permite evaluar cualquier artefacto legible de software. Su objetivo principal es la **detección y eliminación temprana de defectos**, evitando que los errores se propaguen hacia etapas posteriores donde su corrección resultaría mucho más costosa.

Ventajas:

- Detección temprana de defectos y reducción del retrabajo
- Evaluación de versiones o artefactos incompletos
- Análisis de atributos de calidad adicionales que no pueden ser controlados en pruebas automáticas
- Difusión de conocimiento y capacitación cruzada
- Económica y efectiva al momento de encontrar fallas

Desventajas:

- Sobrecarga y costos iniciales elevados
- Dificultad de implementación formal
- Sensación de ralentización del desarrollo

- Consumo de tiempo operacional
- Riesgo de factores humanos y egos, el autor puede percibir los hallazgos como un ataque hacia su persona y adoptar una posición defensiva

---

1. Técnicas de Revisión

A. Recorridas (*Walkthroughs - Técnica Informal*)

- **Mecanismo**: El autor guía informalmente a sus colegas y a las partes interesadas a través del producto de software para que formulen preguntas y señalen posibles errores o incumplimientos de estándares.
- **Características**:
    - Requiere **poca o ninguna preparación previa** y carece de un proceso formal o recolección de métricas.
    - Sus metas principales son la **mínima sobrecarga**, un **rápido retorno** y la **capacitación de los desarrolladores**.
    - Es una de las técnicas más elegidas en marcos ágiles para hacer revisiones rápidas entre pares.

B. Inspecciones *(Técnica Formal)*

- **Mecanismo**: Proceso estructurado y *time-boxed* (reuniones de máximo 2 horas) con roles definidos y un conjunto rígido de etapas.
- **Etapas del Proceso de Inspección**:
    - **Planificación**: El moderador prepara el material, selecciona al equipo y establece el calendario.
    - **Visión General** *(opcional)*: El autor realiza una breve presentación del producto.
    - **Preparación Individual**: Los inspectores analizan el producto por separado para identificar potenciales defectos antes de la reunión.
    - **Reunión de Inspección**: El lector lee el artefacto, el equipo evalúa y filtra los defectos reales y el anotador registra los hallazgos sin ponerse a discutir soluciones.
    - **Corrección**: El autor realiza el retrabajo y corrige los defectos.

    - **Seguimiento**: El moderador verifica las correcciones y se recolectan los datos pertinentes.
- **Roles Requeridos**:
    - **Autor**: Creador del producto y encargado de corregir las fallas.
    - **Moderador**: Planifica, lidera la revisión, coordina la reunión y hace seguimiento de los defectos.
    - **Lector**: Lee o guía la lectura del producto durante la reunión para mantener la concentración del grupo.
    - **Anotador**: Registra formalmente los hallazgos y confecciona el informe final.
    - **Inspector**: Examina el producto con anticipación para encontrar defectos.

---

2. Herramientas y Artefactos de Soporte

Para llevar a cabo las revisiones técnicas de forma sistemática se utilizan los siguientes recursos de apoyo:

- **Listas de Chequeo (*Checklists*)**:
    - Cuestionario estandarizado que guía al inspector sobre los aspectos a controlar
- **Guías de Lectura, Plantillas e Informes**:
    - **Plantillas e Informes de Revisión**: Documentan el resultado y la decisión tomada, los revisores participantes, los defectos hallados y las conclusiones.
    - **Guías de Lectura**: Orientan el análisis individual durante la fase de preparación.
- **Herramientas basadas en Software**:
    - Plataformas especializadas para el soporte electrónico de revisiones, distribución de materiales, registro en línea y seguimiento de hallazgos.
- **Métricas**:
    - **Densidad de defectos**
    - **Total de defectos encontrados**
    - **Esfuerzo de inspección**
    - **Esfuerzo por defecto**
    - **Porcentaje de reinspecciones**

---

3. Reglas y Buenas Prácticas para la Realización

Para garantizar que la revisión sea constructiva y efectiva, se deben aplicar las siguientes pautas esenciales:

1. **Revisar el producto, no al productor**: Evaluar la calidad del artefacto sin juzgar ni evaluar el desempeño individual de las personas.
2. **Fijar una agenda y cumplirla**: Respetar el límite de tiempo y acotar el número de participantes.
3. **Limitar el debate sobre soluciones**: Enunciar y registrar los problemas durante la reunión, pero no tratar de resolverlos en ese momento.
4. **Exigir preparación previa individual**: Asegurar que los inspectores hayan analizado el material detalladamente antes de la reunión grupal.
5. **Tomar notas escritas y capacitar al equipo**: Registrar los hallazgos por escrito, brindar entrenamiento en la técnica y utilizar listas de chequeo consolidadas.

Testing en ambientes Ágiles

El **Testing en Ambientes Ágiles** representa un cambio fundamental de paradigma respecto a los enfoques tradicionales. En el agilismo, las pruebas dejan de ser una fase tardía y aislada al final del proyecto para convertirse en una **actividad continua e integrada** que acompaña al desarrollo en cada iteración o *sprint*.

---

1. El Manifiesto y Principios del Testing Ágil

El enfoque ágil de calidad se fundamenta en los valores del **Manifiesto del Testing**:

- **Pruebas durante todo el desarrollo** por sobre pruebas al final.
- **Prevenir defectos** por sobre encontrarlos.
- **Entender lo que se está probando** por sobre verificar sólo la funcionalidad.
- **Construir un mejor sistema** por sobre romper el sistema.
- **Responsabilidad de todo el equipo por la calidad** por sobre delegar la responsabilidad únicamente al *tester*.

---

2. Las 6 Prácticas Concretas del Testing Ágil

Para materializar este enfoque, los equipos ágiles aplican seis prácticas técnicas centrales:

1. **Desarrollo Conducido por Pruebas (TDD)**: Técnica en la que el desarrollador escribe primero una prueba unitaria que falla (*Red*), implementa el código mínimo para hacerla pasar (*Green*) y luego refactoriza el código (*Refactor*). Garantiza alta cobertura, cohesión y bajo acoplamiento en el diseño del software.
2. **Desarrollo Conducido por Pruebas de Aceptación (ATDD)**: Práctica colaborativa en la que el *Product Owner*, los desarrolladores y los *testers* definen de forma conjunta los criterios de aceptación de las Historias de Usuario mediante *ejemplos ejecutables* antes de codificar.
3. **Pruebas de Unidad e Integración Automatizadas**: Suites de prueba de bajo nivel construidas por los desarrolladores que verifican el correcto funcionamiento de módulos aislados y sus interfaces.
4. **Pruebas de Regresión Automatizadas a Nivel de Sistema**: Ejecución continua y automatizada de pruebas sobre el sistema consolidado a través de *Integración Continua (CI)*, asegurando que los nuevos cambios no introduzcan fallas en funcionalidades existentes.
5. **Pruebas Exploratorias**: Actividades de prueba manual orientadas al aprendizaje en vivo y la creatividad, donde el diseño y la ejecución de casos ocurren simultáneamente para descubrir escenarios no contemplados en las especificaciones.
6. **Control de Versión de Pruebas con el Código**: Los scripts y artefactos de prueba se almacenan y versionan en el mismo repositorio de código fuente.

---

3. Pirámide del Testing: Tradicional vs. Ágil

La estructura del esfuerzo de testing cambia radicalmente entre ambos enfoques:

- **Pirámide Tradicional (Invertida)**: Concentra del **80% al 90%** del esfuerzo en pruebas manuales de interfaz gráfica (GUI) e *End-to-End* al final del proyecto, relegando las pruebas unitarias a solo un **1% - 5%**. Esto genera retroalimentación tardía y altos costos de reparación.
- **Pirámide del Testing Ágil**:
    - **Base ancha (80% - 90%)**: Pruebas unitarias y de componentes automatizadas de rápida ejecución escritas por los desarrolladores.
    - **Capa intermedia (5% - 15%)**: Pruebas automatizadas de reglas de negocio, servicios/APIs y pruebas de aceptación de historias.
    - **Cúspide estrecha (1% - 5%)**: Pruebas de interfaz gráfica (GUI) automatizadas y pruebas manuales/exploratorias enfocadas en la experiencia de usuario.

---

4. Los 4 Cuadrantes del Testing Ágil

Los cuadrantes ayudan a planificar la estrategia de pruebas equilibrando la dimensión técnica con la de negocio:

Cuadrante	Orientación / Foco	Tipo de Pruebas Incluidas	Herramientas / Ejecución
Q1	**Orientadas a la Tecnología / Apoyan al Equipo**	Pruebas Unitarias y Pruebas de Componentes.	Automatizadas (TDD / Frameworks de prueba).
Q2	**Orientadas al Negocio / Apoyan al Equipo**	Pruebas Funcionales, Ejemplos, Pruebas de Historias, Prototipos y Simulaciones.	Automatizadas y Manuales (ATDD).
Q3	**Orientadas al Negocio / Critican el Producto**	Pruebas Exploratorias, Escenarios de Usabilidad/UX, Pruebas de Aceptación de Usuario (UAT) y Monitoreo.	Manuales / Centradas en la experiencia humana.
Q4	**Orientadas a la Tecnología / Critican el Producto**	Pruebas No Funcionales: Rendimiento, Carga, Seguridad, Mantenibilidad, Fiabilidad.	Basadas en Herramientas automáticas de carga y escaneo.

---

5. El Nuevo Rol del Tester en Equipos Ágiles

En los entornos ágiles, el *tester* abandona el rol de "policía de calidad" o *gatekeeper*. Sus funciones principales pasan a ser:

- **Colaborar con el Product Owner**: Ayudando a definir criterios de aceptación claros.
- **Trabajar junto a los Desarrolladores**: Guiando las estrategias de prueba, ayudando a automatizar pruebas de aceptación y notificando hallazgos.
- **Promover la Calidad en el Equipo**: Facilitando la validación del *Definition of Done* y capacitando al equipo para asumir una cultura compartida de calidad.

Retrospectiva

Evento ágil enfocado en la mejora continua, donde el equipo reflexiona sobre su forma de trabajar para identificar problemas y oportunidades de mejora

---

1. Propósito y Fundamento Empírico

- **Pilar del Empirismo**: Es el circuito de retroalimentación dedicado a la **inspección y adaptación** de la forma de trabajar del equipo.
- **Mirar al pasado para mejorar el futuro**: El equipo analiza el periodo transcurrido para entender lo sucedido y aplicar mejoras en la siguiente iteración.
- **Tres preguntas centrales**: El equipo analiza qué salió bien, qué no funcionó y qué se podría mejorar.

---

2. Las 5 Fases de una Retrospectiva

1. **Preparar el escenario**: Establecer el objetivo, la agenda y preparar al equipo para su participación.
2. **Reunir datos:** Se analiza lo ocurrido mediante datos objetivos.
3. **Generar ideas**: El equipo analiza causas y oportunidades de mejora.
4. **Decidir qué hacer**: Se definen propuestas de acción
5. **Cerrar la retrospectiva**: Se reflexiona sobre la sesión, se repasan los compromisos, se agradece y se realiza el cierre.

---

3. Principios y Reglas de Funcionamiento

- Requiere un ambiente de respeto donde no existan acusaciones ni adjudicación de errores.
- El cambio debe nacer del propio equipo de trabajo.
- La retrospectiva no es un espacio para medir o juzgar el rendimiento personal.

---

4. Malos Olores en las Retrospectivas

- Una duración excesivamente corta.
- Respuestas dominadas por los líderes.
- No hablar ni abordar los verdaderos problemas críticos.
- No implementar las acciones de mejora acordadas anteriormente.
- Poca participación
